

```
(defun transicion(color-actual cambiar-a) ;; Cambiar y hacer la validacion del ciclo Verde-amarillo amarillo-rojo r
  (let ((colores_incluidos (list 'en-rojo 'en-amarillo 'en-verde))(colores_cambio (list 'rojo 'amarillo 'verde)))
    (cond
      ((null (member color-actual colores_incluidos)) "El valor de color actual es incorrecto") ;;Agregamos fu
      ((and (member cambiar-a colores_cambio)(list color-actual 'cambiar-a- cambiar-a))) ;;La funcion member
      (t (list color-actual 'accion-por-defecto)) ;; Ver donde se verifica el dato si aca o en alguna entrada
    )
  )
)
```

1. Error de Sintaxis Estructural en el cond:

```
((and (member cambiar-a colores_cambio)(list color-actual 'cambiar-a-
cambiar-a)))
```

El error: Todo está metido dentro de un único gran paréntesis que envuelve a la condición (and ...) y a la lista de retorno (list ...).

Por qué no funciona: Lisp interpreta que todo eso es la condición. Al evaluar el and, si da verdadero, Lisp intentará ejecutar el resultado como si fuera una función (abriendo un nivel de evaluación extra que no existe), rompiendo el flujo del programa o devolviendo algo inesperado en lugar de separar la condición de la acción.

Cómo lo solucionamos:

Separa correctamente la condición de la expresión de retorno:

```
((and (eql 'en-verde color-actual) (eql 'verde-intermitente cambiar-a)) (list
color-actual 'cambiar-a-verde-intermitente))
```

2. Error Semántico: Falta de Validación y Lógica Demasiado Genérica:

El segundo caso del cond original intenta resolver todas las transiciones con una sola línea usando member:

```
(and (member cambiar-a colores_cambio) (list ...))
```

El error: Al usar member, el código dice: "Si el color de destino está en la lista, la transición es válida".

Por qué no funciona: Un semáforo no puede pasar de cualquier color a cualquier color. No valida el "ciclo" o la máquina de estados del semáforo. Por ejemplo, permitiría pasar directamente de en-verde a rojo sin pasar por el amarillo. Además, el símbolo 'cambiar-a-cambiar-a' está escrito de forma literal, por lo que no es dinámico.

Cómo lo solucionamos:

Aplica el principio de especificación: desglosa explícitamente cada una de las transiciones permitidas por el dominio (de verde a verde-intermitente, de verde-intermitente a amarillo, etc.) usando eql para comparar átomo por átomo.

3. Manejo Incorrecto del Flujo del cond y Tipos de Retorno:

```
((null (member color-actual colores_incluidos)) "El valor de color actual es incorrecto")
```

El error: Si el color actual es incorrecto, devuelve un String (una cadena de texto). Sin embargo, si el código cae en el caso por defecto (t), devuelve una Lista (list color-actual 'accion-por-defecto).

Por qué es un problema de diseño: En programación funcional se busca la consistencia en los tipos de retorno. Mezclar strings con listas para el manejo de errores dificulta que otra función pueda procesar ese resultado.

Cómo lo solucionamos:

Si el color actual no es válido, devuelve nil (el equivalente a falso/vacío en Lisp), lo cual es mucho más estándar en este paradigma para indicar un fallo en el control de flujos:

```
((not (member color-actual colores_incluidos)) nil)
```

CÓDIGO CORREGIDO:

```
(defun transicion(color-actual cambiar-a)
```

;;Cambiar y hacer la validacion del ciclo Verde-amarillo amarillo-rojo rojo-verde (hacer validacion inversa)

```
(let ((colores_incluidos (list 'en-rojo 'en-amarillo 'en-verde 'rojo-intermitente 'verde-intermitente 'amarillo-intermitente))
```

```
(colores_cambio (list 'rojo 'amarillo 'verde)))
```

```
(cond
```

```
((not (member color-actual colores_incluidos)) nil)
```

;;Agregamos funcion para verificar que el color-actual este en los permitidos

```
((and (eql 'en-verde color-actual)(eql 'verde-intermitente cambiar-a)(list color-actual 'cambiar-a-verde-intermitente)))
```

```
((and (eql 'verde-intermitente color-actual)(eql 'amarillo cambiar-a)(list color-actual 'cambiar-a-amarillo)))
```

```
((and (eql 'en-amarillo color-actual)(eql 'amarillo-intermitente cambiar-a)(list color-actual 'cambiar-a-amarillo-intermitente)))
```

```
((and (eql 'amarillo-intermitente color-actual)(eql 'rojo cambiar-a)(list color-actual 'cambiar-a-rojo)))
```

```
((and (eql 'en-rojo color-actual)(eql 'rojo-intermitente cambiar-a)(list color-actual 'cambiar-a-rojo-intermitente)))
```

;;La funcion member devolveria NIL en caso de que no se encuentre en la lista definida

```
((and (eql 'rojo-intermitente color-actual)(eql 'verde cambiar-a)(list color-
```

actual 'cambiar-a-verde)))

(t (list color-actual 'accion-por-defecto))

;; Ver donde se verifica el dato si aca o en alguna entrada

;; primer error no verificar el dato antes de devolver color-actual

)

)

)

```
57 ;; *****
58 ;; FUNCION: Duracion del ciclo
59 ;; NATURALEZA: Pura
60 ;; ESTRATEGIA: Calculo aritmetico
61 ;; IMPACTO EN MEMORIA: No destructiva
62 ;; *****
63 (defun duracion-ciclo (&optional (seg-rojo 90)(int-rojo 3)(seg-amarillo 6)(int-amarillo 3)(seg-verde 120)(int-verde 3));; &optional utiliza valores por defecto
64   (unless
65     (every #'(lambda (x)
66               (and (integerp x)
67                    (>= x 0)))
68             (list seg-rojo seg-amarillo seg-verde
69                   int-rojo int-amarillo int-verde))
70     'error-tiempo-invalido)
71   (+ seg-rojo seg-amarillo seg-verde int-rojo int-amarillo int-verde))
72 )
73
74 ;; pero dando la opcion a pasar parametros
```

1. El Error Principal: Error de Flujo y Cortocircuito con unless:

En Common Lisp, la macro unless funciona como un "si no" (el opuesto de when). Su estructura es:

(unless (condicion-falsa)

(expresion1)

(expresion2)

...)

Si la condición es falsa (nil), ejecuta todo el cuerpo que tiene adentro en secuencia y devuelve el resultado de la última expresión. Pero si la condición es verdadera (cualquier cosa distinta de nil), unless no hace nada y devuelve nil.

(defun duracion-ciclo (...)

(unless (every ...) ; Condición de validación

'error-tiempo-invalido) ; Cuerpo del unless

(+ seg-rojo ...) ; ¡Fuera del unless!

)

El error de flujo: La suma de los tiempos (+ seg-rojo ...) quedó completamente afuera de la estructura del unless. En Lisp, una función devuelve implícitamente el valor de la última expresión evaluada en su cuerpo.

Por qué no funciona:

1. Si los datos son válidos, la condición del every es verdadera. El unless se salta (devuelve nil) e inmediatamente se pasa a la línea siguiente, ejecutando la suma. Hasta ahí parece funcionar.
2. Si los datos son inválidos (por ejemplo, pasás un tiempo negativo), la condición del every es falsa. El unless entra a ejecutarse y evalúa el símbolo 'error-tiempo-invalido. Pero como la ejecución de la función continúa de forma secuencial, Lisp pasa a la siguiente expresión: y evalúa la suma de todas formas.
3. El programa intentará sumar los valores incorrectos e ignorará por completo el símbolo de error, rompiendo la lógica de validación defensiva.

2. El Error Semántico de la Resolución con if

La resolución es cambiar el unless por un if.

Al cambiarlo a un if, el flujo queda perfectamente controlado en dos ramas exclusivas:

Rama Verdadera (Datos válidos): Si every es verdadero, ejecuta la suma.

Rama Falsa (Datos inválidos): Si every es falso, ejecuta y devuelve 'error-tiempo-invalido.

Al ser ramas mutuamente excluyentes, es imposible que se ejecute la suma si la validación falla.

CÓDIGO CORREGIDO:

```
(defun duracion-ciclo (&optional (seg-rojo 90)(int-rojo 3)(seg-amarillo 6)
(int-amarillo 3)(seg-verde 120)(int-verde 3))
```

;; &optional utiliza valores por defecto cuando llamas a la funcion sin pasarle parametros

```
(if (every #'(lambda (color)
```

```
(and (integerp color)
```

```
(>= color 0)))
```

```
(list seg-rojo seg-amarillo seg-verde
```

```
int-rojo int-amarillo int-verde))
```

```
(+ seg-rojo seg-amarillo seg-verde
```

```
int-rojo int-amarillo int-verde)
```

```
'error-tiempo-invalido)
```

```
)
```

```
156 )
157
158
159 ;; =====
160 ;; FUNCION: Ciclos por tiempo
161 ;; NATURALEZA: Pura
162 ;; ESTRATEGIA: Composicion funcional, calculo aritmetico y validación condicional
163 ;; IMPACTO EN MEMORIA: No destructiva
164 ;; =====
165 (defun ciclos-por-tiempo (minutos)
166   (if (< minutos 0) 'valor-invalido
167       (nth-value 0(truncate (* minutos 60)(duracion-ciclo))))
168   )
169 )
170
171 ;; =====
```

1. El Error de Tipos: Falta de Validación del Tipo de Dato (integerp)

En Lisp, el operador matemático `<` espera recibir números de manera obligatoria.

Qué pasa en el código original:

La condición original es simplemente: `(< minutos 0)`.

El error: Asume que el argumento `minutos` siempre va a ser un número. Si el usuario llega a pasar por parámetro un dato de otro tipo (por ejemplo, un símbolo como `'hola`, una lista, o un string `"10"`), la función no sabe cómo manejarlo.

Por qué no funciona: Al intentar evaluar `(< 'hola 0)`, el intérprete de Lisp lanzará un error en tiempo de ejecución de tipo `TypeError` (o similar, indicando que el argumento no es un número), rompiendo el programa.

Cómo lo solucionamos:

`(and (integerp minutos) (>= minutos 0))`

Utiliza evaluación por cortocircuito. Primero verifica con `integerp` si el argumento es efectivamente un entero. Si no lo es, el `and` da falso inmediatamente y ni siquiera intenta evaluar la comparación matemática, protegiendo al programa de excepciones imprevistas.

2. El Error Sintáctico: Inversión de la Lógica del if

`(if (< minutos 0)`

`'valor-invalido ; Caso Verdadero`

`(nth-value 0 (truncate ...)) ; Caso Falso`

`)`

El error de lógica: Si la condición original da verdadero (es decir, `minutos` es menor a cero, un valor inválido), la función devuelve `'valor-invalido`. Si

da falso (valores mayores o iguales a cero, valores válidos), calcula los ciclos.

Por qué cambia en la resolución:

Al reescribir la condición en la resolución, se invierte el sentido de la pregunta: ¿Son los minutos un entero mayor o igual a cero? (Pregunta por el caso de éxito/válido).

Por lo tanto, las ramas del if deben invertirse obligatoriamente para mantener la semántica:

```
(if (and (integerp minutos) (>= minutos 0))
```

```
(nth-value 0 (truncate ...)) ;Ahora este es el Caso Verdadero (Éxito)
```

```
'valor-invalido ;Ahora este es el Caso Falso (Error)
```

```
)
```

CÓDIGO CORREGIDO:

```
(defun ciclos-por-tiempo (minutos)
```

```
(if (and (integerp minutos)(>= minutos 0))(nth-value 0(truncate (* minutos 60)(duracion-ciclo)))
```

```
'valor-invalido
```

```
)
```

```
)
```

```
(defun sistema_auditoria(tiempo)
  (let* ((color-actual (timer tiempo))
        (cambiar-a (cond
                     ((eql color-actual 'en-verde) 'amarillo)
                     ((eql color-actual 'en-amarillo) 'rojo)
                     (t 'verde)
                     )
          ))
    (transicion color-actual cambiar-a)
    (format t "Tiempo ~A: la luz ha cambiado de ~A a ~A" tiempo color-actual cambiar-a)
  )
)
```


1. El Error Conceptual: El Paradigma Funcional y los Efectos Secundarios (format)

En Lisp, cuando una función ejecuta múltiples expresiones en secuencia dentro de un `let*`, devuelve implícitamente el valor de la última expresión evaluada.

```
(transicion color-actual cambiar-a)
```

```
(format t "Tiempo ~A: la luz ha cambiado..." tiempo color-actual cambiar-a)
```

El error: La función `transicion` (que calcula y devuelve la lista con la transición del semáforo) se ejecuta en penúltimo lugar. Justo después, se ejecuta la función `format` con el parámetro `t` (que significa imprimir por consola/salida estándar).

Por qué no funciona: En Common Lisp, la función `format t ...` siempre devuelve `nil` tras imprimir el texto en pantalla. Como es la última línea, toda la función `sistema_auditoria` termina devolviendo `nil`. El cálculo real de la transición se "pierde" en la ejecución.

2. El Error de Dominio: El Ciclo de Transición Incorrecto

Qué pasa en el código original:

El `cond` original asume una secuencia de luces muy simple:

Si es en-verde `\rightarrow` pasa a amarillo.

Si es en-amarillo `\rightarrow` pasa a rojo.

Sino, pasa a verde.

El error de lógica: Este ciclo ignora por completo los estados intermitentes de transición (rojo-intermitente, amarillo-intermitente, verde-intermitente) que el sistema del semáforo requiere (como vimos en los ejercicios anteriores). Es un análisis incompleto del dominio del problema.

Cómo lo solucionamos:

La resolución desglosa de manera exhaustiva y correcta la máquina de estados completa del semáforo real, mapeando cada luz con su estado intermitente previo correspondiente.

3. Error de Acoplamiento y Manejo del Tiempo

Qué pasa en el código original:

La función original recibe un parámetro llamado tiempo y depende de una función interna (timer tiempo).

El error de acoplamiento: Está diseñada para un tiempo estático o simulado que se le pasa por parámetro, lo que no sirve para una auditoría real del sistema en tiempo de ejecución.

Cómo lo solucionamos:

La resolución elimina el parámetro de la función (defun sistema-auditoria () ...) y calcula el tiempo de manera autónoma y dinámica usando una primitiva del sistema:

(- (get-universal-time) 2208988800)

get-universal-time devuelve los segundos transcurridos desde el 1 de enero de 1900. Al restarle 2208988800 (que son los segundos entre 1900 y 1970), se está haciendo una conversión manual al Tiempo Unix (Epoch) para sincronizar el reloj con sistemas modernos.

CÓDIGO CORREGIDO:

(defun sistema-auditoria ()

(let* (

(tiempo-actual (- (get-universal-time) 2208988800))

(color-nuevo (timer_con_intermitencia tiempo-actual))

(color-anterior

(cond

((eq color-nuevo 'en-verde) 'rojo-intermitente)

((eq color-nuevo 'rojo-intermitente) 'en-rojo)

((eq color-nuevo 'en-rojo) 'amarillo-intermitente)

((eq color-nuevo 'amarillo-intermitente) 'en-amarillo)

((eq color-nuevo 'verde-intermitente) 'en-verde)

(t 'verde-intermitente)

)

)

)

```
;;=====
=====
REQUERIMIENTO 6: Informe de Distribución
Temporal ;; FUNCIÓN: Porcentaje ;;
NATURALEZA: Pura ;; ESTRATEGIA: Calculo
aritmético ;; IMPACTO: No destructivo ;;
=====
=====
```

```
(defun porcentaje(tiempo-color total-ciclo) ;
Total del ciclo rojo 90 + amarillo 6 + verde
120 = 216 seg (* (/ tiempo-color (float total-
ciclo)) 100) )
```

1. El Error Sintáctico: Comentarios que "Mutilan" el Código

```
(defun porcentaje(tiempo-color total-ciclo) ; Total del ciclo rojo 90 +
amarillo 6 + verde 120 = 216 seg (* (/ tiempo-color (float total-ciclo)) 100)
))
```

Por qué no funciona: El programador original escribió el comentario explicativo, pero dejó la fórmula matemática $(* (/ \text{tiempo-color } \dots))$ y los paréntesis de cierre de la función en la misma línea, justo después del punto y coma. Como consecuencia, para el entorno de ejecución, el cuerpo de la función quedó completamente vacío. Al intentar compilar o evaluar la función, Lisp arrojará un error de paréntesis no balanceados (faltan cerrar los de defun) y la lógica del cálculo matemático directamente no existirá.

Cómo lo solucionamos:

Mueve el cálculo a una línea separada (o remueve la parte comentada que pisaba el código), asegurando que la expresión S (S-expression) sea correctamente interpretada por el entorno.

2. El Error Semántico y de Dominio: Omisión de Variables del Ciclo

Qué pasa en el código original:

En el comentario original se lee: Total del ciclo rojo 90 + amarillo 6 + verde 120 = 216 seg.

El error de lógica del problema: El desarrollador calculó el total sumando solo los tiempos principales de los semáforos, pero se olvidó de sumar los tiempos de los intervalos intermitentes (que vimos en los ejercicios anteriores, donde sumaban 3 segundos por cada transición).

Cómo lo solucionamos:

El comentario de la resolución corrige este entendimiento del problema (el modelo de dominio): rojo 90 + 3, amarillo 6 + 3, verde 120 + 3 = 225 seg. Si bien esto es parte de la documentación en el comentario, demuestra que el cálculo del parámetro total-ciclo que se le envíe a la función debe contemplar la totalidad real de las variables para evitar un error de precisión en los porcentajes calculados.

3. Ausencia de Validación Defensiva (Tipado Dinámico)

Al igual que en el ejercicio anterior, el código original carece de validación para tiempo-color y total-ciclo.

Si total-ciclo llegara a ser un cero (0), el código original intentaría hacer una división por cero (/ tiempo-color 0), provocando una excepción crítica de división por cero (ZeroDivisionError) que colgaría la ejecución. Al incluir el if con (integerp ...) y (>= ... 0) de forma estricta, el código se vuelve robusto y seguro.

CÓDIGO CORREGIDO:

(defun porcentaje(tiempo-color total-ciclo) ; Total del ciclo rojo 90 + 3 ,
amarillo 6 + 3, verde 120 + 3 = 225 seg

(if (and (integerp tiempo-color)(>= tiempo-color 0)(integerp total-ciclo)(>=
total-ciclo 0))(* (/ tiempo-color (float total-ciclo)) 100)

'valor-invalido

)

)